

PySpark Introduction

Software installed.

Java 17 or 24

Python 3

PySpark is the Python API for Apache Spark.

It allows you to process large-scale data across multiple machines using Python.

Why PySpark?

Handles Big Data efficiently

Distributed processing

Fault tolerant

In-memory computation

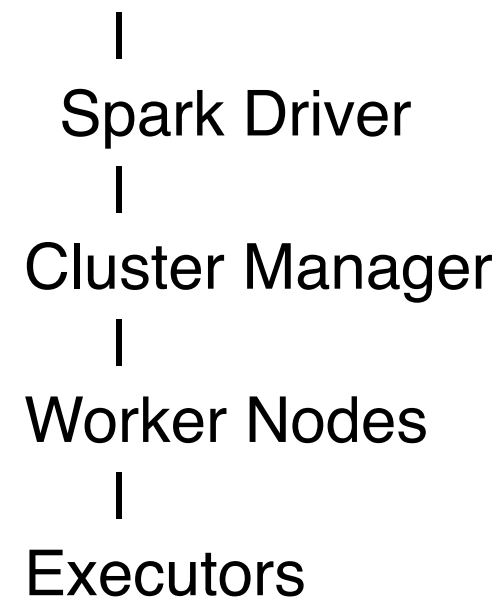
Supports SQL, Streaming, ML, Graph Processing

Architecture

Python Application

|

PySpark



Common Components

SparkSession → Entry point

RDD → Low-level distributed data structure

DataFrame → Structured data with schema

Spark SQL → SQL queries on data

MLlib → Machine Learning

Structured Streaming → Real-time processing

Create Spark Session

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder \
    .appName("MyApp") \
    .getOrCreate()
```

What is DAG?

DAG = Directed Acyclic Graph

Spark does not execute operations immediately.

Instead, it creates an execution plan called a DAG.

RDD/DataFrame

```
|
filter
|
map
|
groupBy
|
```

count

Spark analyzes the complete workflow before execution and optimizes it.

DAG Terminology

Directed

Operations have a direction.

Read File



Filter



GroupBy

Acyclic

No loops allowed.

✓ Valid

$A \rightarrow B \rightarrow C$

✗ Invalid

$A \rightarrow B \rightarrow C$
↑ ↓
└──────────┘

Lazy Evaluation

Transformations are not executed immediately.

```
df = spark.read.csv("employees.csv")
```

```
filtered = df.filter(df.salary > 50000)
```

```
result = filtered.select("name")
```

No execution yet.

Execution starts only when an Action is called.

```
result.show()
```

At this point Spark builds the DAG and executes it.

Example DAG

Code:

```
df = spark.read.csv("employees.csv")
```



```
result = df.filter(df.salary > 50000) \
    .groupBy("department") \
    .count()
```

```
result.show()
```

DAG:

Read CSV

|

Filter

|

GroupBy

|

Count

|

Show

Stages and Tasks

Spark breaks the DAG into:

Stage

A collection of operations executed together.

Stage 1:

Read → Filter

Stage 2:

GroupBy → Count

Task

Smallest execution unit.

If there are 4 partitions:

Stage 1

├─ Task 1
├─ Task 2
├─ Task 3
└─ Task 4

Each task runs on an executor.

Narrow vs Wide Transformation

Narrow Transformation

No data movement between partitions.

Examples:

map()
filter()
select()

Partition1 → Partition1
Partition2 → Partition2

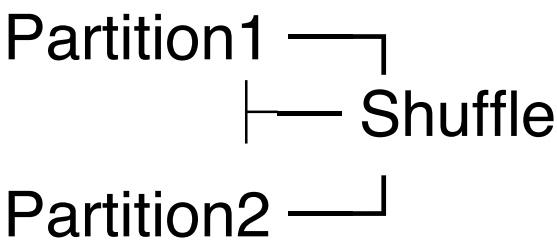
Fast.

Wide Transformation

Requires shuffle.

Examples:

groupBy()
join()
distinct()
reduceByKey()



Expensive operation.

Interview Questions

What is DAG in Spark?

A DAG (Directed Acyclic Graph) is Spark's execution plan that represents transformations and dependencies before execution.

Why is DAG important?

Optimizes execution

Minimizes data movement

Improves performance

Enables fault tolerance

When is DAG created?

When an Action (`show`, `count`, `collect`) is triggered.

What is Lazy Evaluation?

Transformations are stored in the DAG and executed only when an Action is called.

Which transformations create new stages?

Wide transformations such as:

groupBy()
join()
reduceByKey()
distinct()

These cause shuffling and stage boundaries.

30-Minute Practice

1. Read a CSV file.
2. Apply `filter()`.
3. Apply `select()`.
4. Apply `groupBy().count()`.
5. Call `explain()` and observe the execution plan.

6. Compare a simple `filter()` DAG vs a `join()` DAG.
7. Identify which operations are narrow and wide transformations.

```
result.explain(True)
```

This command is one of the best ways to understand how Spark builds and optimizes a DAG internally.

```
# Pandas  
df[df["salary"] > 50000]
```

```
# PySpark  
df.filter(df.salary > 50000)
```

But the big difference is scale and execution.

RDD vs DataFrame

Feature	RDD	DataFrame
Data Structure	Objects	Rows & Columns
Schema	No	Yes
SQL Support	No	Yes
Optimization	No	Catalyst Optimizer
Performance	Slower	Faster
Ease of Use	Harder	Easier

Example

Suppose you have:

10,000 rows → Pandas is perfect.

10 million rows → Pandas may struggle.

1 billion rows → PySpark is designed for this.

Pandas


```
df = pd.read_csv("employees.csv")
```

Entire file loads into your laptop RAM.

If the file is 20 GB and your laptop has 16 GB RAM:

MemoryError

PySpark

```
df = spark.read.csv("employees.csv")
```

Data is split into partitions:

Machine 1 -> 5 GB

Machine 2 -> 5 GB

Machine 3 -> 5 GB

Machine 4 -> 5 GB

All machines process in parallel.

Why DAG Matters

In Pandas:

```
df = pd.read_csv(...)
df = df[df.salary > 50000]
df = df[["name"]]
```

Each line executes immediately.

In PySpark:

```
df = spark.read.csv(...)
df = df.filter(df.salary > 50000)
df = df.select("name")
```


Nothing runs yet.

Spark builds a DAG:

Read CSV

|

Filter

|

Select

Only when:

`df.show()`

Spark optimizes the plan and executes it.

most important **PySpark** methods with simple definitions:

Method	Definition
show()	Displays DataFrame records
printSchema()	Displays column names and data types
count()	Returns total number of rows
select()	Selects specific columns
filter()	Filters rows based on a condition
where()	Same as filter()
distinct()	Removes duplicate rows
dropDuplicates()	Removes duplicate records
orderBy()	Sorts data
sort()	Sorts data
groupBy()	Groups records for aggregation
agg()	Performs aggregate calculations
withColumn()	Adds or modifies a column
withColumnRenamed()	Renames a column
drop()	Removes a column
join()	Combines two DataFrames
union()	Combines rows from two DataFrames

<code>limit()</code>	Returns first N rows
<code>describe()</code>	Generates summary statistics
<code>createOrReplaceTempView()</code>	Creates a temporary SQL table
<code>explain()</code>	Shows execution plan (DAG)
<code>cache()</code>	Stores DataFrame in memory
<code>persist()</code>	Stores DataFrame in memory/disk
<code>collect()</code>	Returns all rows to driver
<code>first()</code>	Returns first row
<code>head()</code>	Returns top rows
<code>sample()</code>	Returns random sample data
<code>repartition()</code>	Changes number of partitions
<code>coalesce()</code>	Reduces partitions
<code>write.csv()</code>	Writes DataFrame to CSV
<code>write.parquet()</code>	Writes DataFrame to Parquet
<code>write.jdbc()</code>	Writes DataFrame to Database
<code>read.csv()</code>	Reads CSV file
<code>read.parquet()</code>	Reads Parquet file
<code>read.json()</code>	Reads JSON file
<code>read.jdbc()</code>	Reads data from Database

Common Date Methods

Method	Definition
to_date()	Converts string to date
year()	Extracts year
month()	Extracts month
dayofmonth()	Extracts day
current_date()	Returns current date
datediff()	Calculates date difference

Common RDD Methods

Method	Definition
map()	Transforms each record
flatMap()	Flattens nested records
filter()	Filters records
reduce()	Aggregates records into a single value
collect()	Returns all records
count()	Counts records
take()	Returns first N records

Interview Definitions

select()

Used to retrieve specific columns from a DataFrame.

filter()

Used to retrieve rows matching a condition.

groupBy()

Used to group data and perform aggregations such as count, sum, and average.

join()

Used to combine data from multiple DataFrames based on a common column.

withColumn()

Used to create a new column or modify an existing column.

explain()

Used to display the logical and physical execution plan of a Spark job.

repartition()

Used to increase or redistribute partitions for parallel processing.

cache()

Used to store frequently used DataFrames in memory to improve performance.

These are the methods most commonly used in PySpark interviews and real-world ETL/Data Engineering projects.

File Types PySpark Can Process

File Type	Extension
CSV	.csv
JSON	.json
Parquet	.parquet
ORC	.orc
Avro	.avro
Text	.txt
XML	.xml (with package)
Excel	.xlsx (with package)
Delta Lake	.delta
Log Files	.log

Most Common in Real Projects

CSV → 20%

JSON → 20%

Parquet → 40%

Database → 15%

Kafka → 5%

Type	Example
StringType	"John"
IntegerType	100
LongType	100000
DoubleType	10.5
BooleanType	True
DateType	2026-06-18
TimestampType	2026-06-18 10:30:00
ArrayType	["Java","Python"]
StructType	{"city":"Chennai"}

MapType	{"Java":5}
---------	------------

PySpark can process almost any alphanumeric data, whether structured, semi-structured, or unstructured.

What PySpark cannot directly process

Images (.jpg, .png)

Audio (.mp3)

Video (.mp4)

PDF binary files

It can **read them as binary files**, but it doesn't understand image/audio/video content natively.

Pandas

↓

Good for small text datasets

PySpark



Good for millions of paragraphs/documents

For example:

10 customer reviews	→ Pandas
10 million reviews	→ PySpark
100 GB text corpus	→ PySpark